

Module : Programming with Rust

TP : TP2 – File Integrity Checker and IOC Matcher in Rust.

Étudiant : Zeinebou Elhachmi

Matricule : 22048

Année universitaire : 2025–2026

GitHub Repository :

https://github.com/Chew9i/cd-tp2_integrity_checker.git

Required Docker Compose environment

[illegible]

```
PS C:\Users\lapto\Documents\cyber\s2\Programming with Rust\rust-git\compose-recipes\rust-venv\workspace> docker compose exec rustlab bash
HP_LAPTOP@f800e706bccb:/workspace$ ls
exercises  filecheck  tid2_ownership  tid4_alert_records  tid5_collections_crypto  tpi_log_analyzer
HP_LAPTOP@f800e706bccb:/workspace$ rustc --version
rustc 1.94.1 (e408947bf 2026-03-25)
cargo --version
cargo 1.94.1 (29ea6fb6a 2026-03-24)
cargo fmt --version
rustfmt 1.8.0-stable (e408947bfd 2026-03-25)
cargo clippy --version
clippy 0.1.94 (e408947bfd 2026-03-25)
cargo audit --version
cargo-audit-audit 0.22.1
cargo deny --version
cargo-deny 0.19.8
HP_LAPTOP@f800e706bccb:/workspace$
```

```
HP_LAPTOP@f800e706bccb:/workspace$ cargo new tp2_integrity_checker
  Creating binary (application) `tp2_integrity_checker` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
HP_LAPTOP@f800e706bccb:/workspace$ cd tp2_integrity_checker
HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$
```

Required dependency

```

Cargo.toml U
Cargo.toml
1  [package]
2  name = "tp2_integrity_checker"
3  version = "0.1.0"
4  edition = "2024"
5
6  [dependencies]
7  sha2 = "0.10"
```

```
HP_LAPTOP@f800e706bccb:/workspace$ cd tp2_integrity_checker
HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$ cargo check
    Checking tp2_integrity_checker v0.1.0 (/workspace/tp2_integrity_checker)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 4.94s
HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$
```

Sample input data

```

    Finished `dev` profile [unoptimized + debuginfo] target(s) in 4.94s
HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$ mkdir -p samples/files
HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$ printf "Welcome to TP2 Rust integrity lab.\n" > samples/files/clean_readme.txt
HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$ printf "MALWARE-TEST-SAMPLE-TP2\n" > samples/files/suspicious_dropper.txt
HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$ printf "Normal administrative notes.\n" > samples/files/notes.txt
HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$
```

```

HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$ sha256sum samples/files/*
70bbeaa0f2d408a45827aa9d5bd58209564a5bd7b61d6c069267c9e9e35f97cd  samples/files/clean_readme.txt
1888673b4c962129e57b54b81fda2f967c52f87c28e9d7908e5fba0dfed097e3  samples/files/notes.txt
44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b  samples/files/suspicious_dropper.txt
HP_LAPTOP@f800e706bccb:/workspace/tp2_integrity_checker$
```

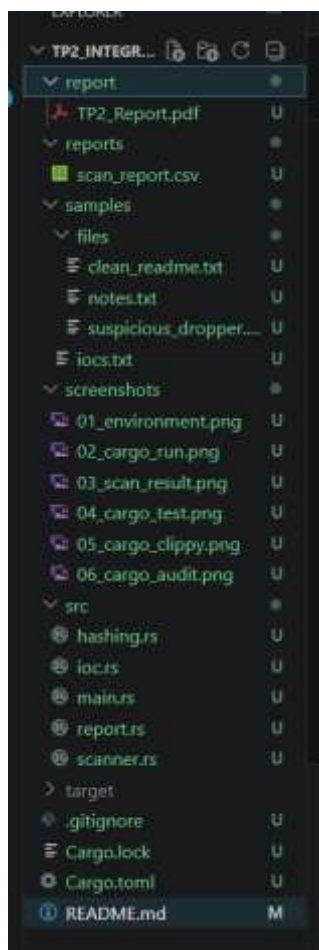
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SEARCH
GNU nano 7.2 samples/iocs.txt *
# hash,label

44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b,Demo suspicious test sample

aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa,Fake known malware hash
[
INVALID_HASH_LINE_SHOULD_BE_IGNORED
```

Implementation

Project structure :



Codes :

src/main.rs

```
Cargo.toml U  main.rs U X  report.rs U  scanner.rs U  ioc.rs U  hashing.rs U
src > main.rs
1  mod hashing;
2  mod ioc;
3  mod report;
4  mod scanner;
5
6  use std::env;
7  use std::path::Path;
8
9  use ioc::load_iocs;
10 use report::write_csv_report;
11 use scanner::{ScanStatus, scan_target};
12
13 fn main() {
14     let args: Vec<String> = env::args().collect();
15
16     if args.len() != 7 {
17         println!(
18             "Usage:
19 tp2_integrity_checker --target <FILE_OR_DIRECTORY> --ioc <IOC_FILE> --report <REPORT_FILE>"
20         );
21         return;
22     }
23
24     let target = Path::new(&args[2]);
25     let ioc_file = Path::new(&args[4]);
26     let report_file = Path::new(&args[6]);
27
28     println!("TP2 File Integrity Checker and IOC Matcher");
29     println!("Target: {}", target.display());
30     println!("IOC file: {}", ioc_file.display());
31     println!("Report: {}", report_file.display());
32
33     let (iocs, invalid_iocs) = match load_iocs(ioc_file) {
34         Ok(data) => data,
35         Err(e) => {
36             eprintln!("Error loading IOC file: {}", e);
37             return;
38         }
39     };
40 }
```

```

let results = scan_target(target, &iocs);

let matches: Vec<_> = results
    .iter()
    .filter(|r| matches!(r.status, ScanStatus::Match(_)))
    .collect();

println!("\nSummary:");
println!("* Files scanned: {}", results.len());
println!("* IOC entries loaded: {}", iocs.len());
println!("* Invalid IOC lines: {}", invalid_iocs);
println!("* Matches found: {}", matches.len());
println!("* Errors: 0");

if !matches.is_empty() {
    println!("\nMatches:");

    for m in &matches {
        if let ScanStatus::Match(label) = &m.status {
            println!("[ALERT] {}", m.path);
            println!("SHA-256: {}", m.sha256.clone().unwrap_or_default());
            println!("IOC label: {}", label);
        }
    }
}

if let Err(e) = write_csv_report(&results, report_file) {
    eprintln!("Failed to write report: {}", e);
    return;
}

println!("\nCSV report written to {}", report_file.display());
}

```

src/report.rs

```
src > report.rs
1  use crate::scanner::{ScanResult, ScanStatus};
2
3  use std::fs::{File, create_dir_all};
4  use std::io::{self, Write};
5  use std::path::Path;
6
7  pub fn write_csv_report(results: &[ScanResult], report_path: &Path) -> io::Result<()> {
8      if let Some(parent) = report_path.parent() {
9          create_dir_all(parent)?;
10     }
11
12     let mut file = File::create(report_path)?;
13
14     writeln!(file, "path,sha256,status,label")?;
15
16     for result in results {
17         let (status, label) = match &result.status {
18             ScanStatus::Clean => ("CLEAN", String::new()),
19             ScanStatus::Match(label) => ("MATCH", label.clone()),
20             ScanStatus::Error(err) => ("ERROR", err.clone()),
21         };
22
23         writeln!(
24             file,
25             "{},{},{},{}",
26             result.path,
27             result.sha256.clone().unwrap_or_default(),
28             status,
29             label
30         )?;
31     }
32
33     Ok(())
34 }
35
```

src/scanner.rs

```
src > scanner.rs
1  use crate::hashing::hash_file_sha256;
2  use crate::ioc::IocEntry;
3
4  use std::fs;
5  use std::path::Path;
6
7  #[derive(Debug, Clone, PartialEq, Eq)]
8  pub enum ScanStatus {
9      Clean,
10     Match(String),
11     Error(String),
12 }
13
14 #[derive(Debug, Clone, PartialEq, Eq)]
15 pub struct ScanResult {
16     pub path: String,
17     pub sha256: Option<String>,
18     pub status: ScanStatus,
19 }
20
21 pub fn scan_target(target: &Path, iocs: &[IocEntry]) -> Vec<ScanResult> {
22     let mut results = Vec::new();
23
24     if target.is_file() {
25         scan_file(target, iocs, &mut results);
26     } else if target.is_dir() {
27         if let Ok(entries) = fs::read_dir(target) {
28             for entry in entries.flatten() {
29                 let path = entry.path();
30
31                 if path.is_file() {
32                     scan_file(&path, iocs, &mut results);
33                 }
34             }
35         }
36     }
37
38     results
39 }
40
```



```

> scanner.rs
9  }
10
11 fn scan_file(path: &Path, iocs: &[IocEntry], results: &mut Vec<ScanResult>) {
12     match hash_file_sha256(path) {
13         Ok(hash) => {
14             let mut status = ScanStatus::Clean;
15
16             for ioc in iocs {
17                 if ioc.hash == hash {
18                     status = ScanStatus::Match(ioc.label.clone());
19                     break;
20                 }
21             }
22
23             results.push(ScanResult {
24                 path: path.display().to_string(),
25                 sha256: Some(hash),
26                 status,
27             });
28         }
29
30         Err(e) => {
31             results.push(ScanResult {
32                 path: path.display().to_string(),
33                 sha256: None,
34                 status: ScanStatus::Error(e.to_string()),
35             });
36         }
37     }
38 }
39

```

src/ioc.rs

```
c > ioc.rs
1 use std::fs;
2 use std::io;
3 use std::path::Path;
4
5 #[derive(Debug, Clone, PartialEq, Eq)]
6 pub struct IocEntry {
7     pub hash: String,
8     pub label: String,
9 }
10
11 pub fn is_valid_sha256(value: &str) -> bool {
12     value.len() == 64 && value.chars().all(|c| c.is_ascii_hexdigit())
13 }
14
15 pub fn load_iocs(path: &Path) -> Result<Vec<IocEntry>, usize>, io::Error> {
16     let content = fs::read_to_string(path)?;
17
18     let mut iocs = Vec::new();
19     let mut invalid_count = 0;
20
21     for line in content.lines() {
22         let line = line.trim();
23
24         if line.is_empty() || line.starts_with('#') {
25             continue;
26         }
27
28         let parts: Vec<&str> = line.splitn(2, ',').collect();
29
30         if parts.len() != 2 {
31             invalid_count += 1;
32             continue;
33         }
34
35         let hash = parts[0].trim();
36         let label = parts[1].trim();
37     }
38 }
```

```

3         if is_valid_sha256(hash) {
4             iocs.push(IocEntry {
5                 hash: hash.to_string(),
6                 label: label.to_string(),
7             });
8         } else {
9             invalid_count += 1;
10        }
11    }
12
13    Ok((iocs, invalid_count))
14}
15
16#[cfg(test)]
17mod tests {
18    use super::*;
19
20    #[test]
21    fn test_valid_sha256() {
22        assert!(is_valid_sha256(
23            "44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b"
24        ));
25    }
26
27    #[test]
28    fn test_invalid_sha256() {
29        assert!(!is_valid_sha256("invalid_hash"));
30    }
31}

```

src/hashing.rs

```
src > hashing.rs
1  use sha2::{Digest, Sha256};
2  use std::fs;
3  use std::io;
4  use std::path::Path;
5
6  pub fn hash_file_sha256(path: &Path) -> Result<String, io::Error> {
7      let data = fs::read(path)?;
8
9      let mut hasher = Sha256::new();
10     hasher.update(&data);
11
12     Ok(format!("{:x}", hasher.finalize()))
13 }
14
15 #[cfg(test)]
16 mod tests {
17     use super::*;
18
19     #[test]
20     fn test_valid_hash_length() {
21         let hash = "a".repeat(64);
22         assert_eq!(hash.len(), 64);
23     }
24 }
25
```

IOC :

```
samples > locs.txt
1  # hash,label
2
3  44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b,Demo suspicious test sample
4
5  aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa,Fake known malware hash
6
7  INVALID_HASH_LINE_SHOULD_BE_IGNORED
8
```

CSV Report :

```
reports > scan_report.csv
1 path,sha256,status,label
2 samples/files/clean_readme.txt,70bbeaa0f2d408a45827aa9d5bd58209564a5bd7b61d6c069267c9e9e35f97cd,CLEAN,
3 samples/files/notes.txt,1888673b4c962129e57b54b81fda2f967c52f87c28e9d7908e5fba0dfed097e3,CLEAN,
4 samples/files/suspicious_dropper.txt,44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b,MATCH,Demo suspicious test sam
5
```

Execution Evidence

cargo fmt

cargo test

```
HP_LAPTOP@6d8bf5864c14:/workspace/tp2_integrity_checker$ cargo fmt
HP_LAPTOP@6d8bf5864c14:/workspace/tp2_integrity_checker$ cargo test
warning: unused import: `super::*`
--> src/hashing.rs:17:9
17 |     use super::*;
    |     ^^^^^^^^^
= note: `[warn(unused_imports)]` (part of `[warn(unused)]`) on by default

warning: `tp2_integrity_checker` (bin "tp2_integrity_checker" test) generated 1 warning (run `cargo fix --bin "tp2_integrity_checker"
sts` to apply 1 suggestion)
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.37s
Running unittests src/main.rs (target/debug/deps/tp2_integrity_checker-ef6558d79bb582ba)

running 3 tests
test ioc::tests::test_valid_sha256 ... ok
test ioc::tests::test_invalid_sha256 ... ok
test hashing::tests::test_valid_hash_length ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

HP_LAPTOP@6d8bf5864c14:/workspace/tp2_integrity_checker$
```

```
cargo run -- --target samples/files --ioc samples/iocs.txt --report reports/scan_report.csv
```

```
HP_LAPTOP@6d8bf5864c14:/workspace/tp2_integrity_checker$ cargo run -- --target samples/files --ioc samples/iocs.txt --report reports/scan_report.csv
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.57s
    Running `target/debug/tp2_integrity_checker --target samples/files --ioc samples/iocs.txt --report reports/scan_report.csv`
TP2 File Integrity Checker and IOC Matcher
Target: samples/files
IOC file: samples/iocs.txt
Report: reports/scan_report.csv

Summary:
* Files scanned: 3
* IOC entries loaded: 2
* Invalid IOC lines: 1
* Matches found: 1
* Errors: 0

Matches:
[ALERT] samples/files/suspicious_dropper.txt
SHA-256: 44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b
IOC label: Demo suspicious test sample

CSV report written to reports/scan_report.csv
HP_LAPTOP@6d8bf5864c14:/workspace/tp2_integrity_checker$
```

Bonus tasks

```
HP_LAPTOP@6d8bf5864c14:/workspace/tp2_integrity_checker$ cargo audit --version
cargo-audit-audit 0.22.2
HP_LAPTOP@6d8bf5864c14:/workspace/tp2_integrity_checker$
```

cargo audit

```
cargo-audit-audit 0.22.2
HP_LAPTOP@6d8bf5864c14:/workspace/tp2_integrity_checker$ cargo audit
    Fetching advisory database from `https://github.com/RustSec/advisory-db.git`
    Loaded 1131 security advisories (from /opt/rust/cargo/advisory-db)
    Updating crates.io index
    Scanning Cargo.lock for vulnerabilities (11 crate dependencies)
HP_LAPTOP@6d8bf5864c14:/workspace/tp2_integrity_checker$
```

Bonus Task – Cargo Audit

As an optional bonus task, a dependency security audit was performed using the cargo audit tool. The audit checked all dependencies listed in the project's Cargo.lock file against the RustSec advisory database.

The scan completed successfully and no known security vulnerabilities were detected in the project's dependencies. This verification provides additional assurance regarding the security of the external crates used by the application.

Discussion

Invalid IOC Handling

The application was designed to handle invalid IOC entries safely. During the parsing process, empty lines, comment lines, and malformed IOC entries are ignored. Invalid IOC lines are counted and reported in the final summary without interrupting the execution of the program. This approach improves robustness and prevents crashes caused by incorrect input data.

Missing File Handling

The program uses Rust's error handling mechanisms to detect missing files or directories. Instead of terminating unexpectedly, the application displays a meaningful error message to the user. This behavior improves usability and reliability when incorrect paths are provided.

Why SHA-256 is Preferred over MD5 or SHA-1

SHA-256 is considered a secure cryptographic hash algorithm and is widely used in cybersecurity applications. Older algorithms such as MD5 and SHA-1 are vulnerable to collision attacks, meaning that different files can produce the same hash value. Because of these weaknesses, SHA-256 provides a higher level of integrity verification and is recommended for modern security tools.

Possible Improvements

Several improvements could be added to this project in future versions. Recursive directory scanning would allow the tool to analyze files in subdirectories automatically. Additional output formats such as JSON could improve interoperability with other tools. Performance could be enhanced through parallel file scanning using Rust threads. Finally, integration tests and support for larger IOC databases could make the application more suitable for real-world security environments.

Conclusion

This project successfully implemented a File Integrity Checker and IOC Matcher in Rust. The application calculates SHA-256 hashes for files, loads and validates IOC entries, compares file hashes against known indicators of compromise, and generates a CSV report summarizing the scan results.

Through this project, several important Rust concepts were practiced, including file handling, error management using Result, data structures, enumerations, vectors, modular programming, and unit testing. The project also demonstrated the use of external crates and secure programming practices.

Overall, the TP provided practical experience in defensive security programming while reinforcing fundamental Rust development skills.